

OVERLAYCCL: Composing Collectives Above a Vendor Library via a Closed-Stack Search Loop

Anonymous Author(s)

Abstract

Picking the fastest collective-communication strategy for a distributed-training workload is hard. Timing each strategy on a microbenchmark is cheap but can pick a strategy that is slow or fails to compile inside a real training step. Running each strategy end-to-end is direct, but realistic GPU-cluster experiments are expensive. Cost-model simulators have been explored, but existing ones are tuned against a fixed reference environment, so their internal numbers drift across machines and model shapes. The simulator approach is even harder to get right on closed-source vendor stacks (AWS Trainium, Google TPU, Meta MTIA), because of two specific properties: (C1) *no visibility into the runtime* — when an operation is slow the user cannot inspect the vendor library to see why — and (C2) *no low-level control* — vendor primitives are called through a fixed API, with no exposed schedule-level intermediate representation (which would let a user choose how messages are routed inside a collective communication call).

We propose OVERLAYCCL, a closed-stack search loop that addresses each of these challenges. For (C1), an LLM agent builds the cost-model simulator on the target deployment itself: it writes probing code that measures the simulator’s internal numbers against the actual device and model shape, so the simulator’s estimates re-adapt to the deployment when either changes. For (C2), we accept the API as fixed and search one layer above it: which primitives to call, in what order, and how to reshape, pad, or slice the input/output tensors around each call. Against an internal-AWS-optimized baseline used in production on Trainium today, OVERLAYCCL delivers a **1.40× speedup** on OLMoE-10B at actual training runs — a 29% reduction in end-to-end steady-step time, with matched descent on real OpenWebText — and a **3.24× speedup** on Llama-7B. Per-primitive sweeps across the 8 collective problems yield individual speedups from 1.27× to 11.41×; the end-to-end speedups generalize to cluster sizes from 3 to 7 nodes. A multi-seed retest at n at least 5 seeds yields clean agent wins (twice the standard error of the difference) on every primitive, with loss floors bit-identical across seeds.

Keywords: collective communication, distributed training, LLM agents, AWS Trainium, cost-model simulator, closed-stack accelerators

1 Introduction

Choosing the fastest collective-communication strategy for a distributed-training workload is challenging, and two prevailing methods have substantial limitations. Running each candidate strategy end-to-end on the actual workload is direct but costly: realistic GPU-cluster experiments are expensive, and the cost grows with the number of strategies tried. Ranking strategies by a cheap microbenchmark on small isolated calls is much cheaper, but on a Trainium cluster we show that multiple strategies that win microbenchmarks are materially *slower* at actual training runs than a different strategy that loses in microbenchmarks (Figure 2). We call this *cross-scope inversion*: moving an operation from an isolated microbenchmark into a real multi-rank training step changes the per-call cost, because the surrounding training graph changes how the runtime launches, fuses, and lays out tensors for the call.

A third approach, scoring strategies against a cost-model simulator, has been explored in prior work (SimAI [61], ASTRA-sim [43, 63]). But existing simulators are written and tuned against a fixed reference configuration: the numbers they use internally — per-collective bandwidth, the overhead each time the runtime launches a graph, when adjacent calls fuse, etc. — drift across machines and model shapes, and re-tuning them by hand for each new environment or training task is again costly. The simulator-based approach gets even harder on the closed-source vendor stacks production training increasingly runs on (AWS Trainium, Google TPU, Meta MTIA), because of two specific properties of these stacks. **(C1) No visibility into the runtime:** when an operation (e.g., `xm.all_to_all` from Trainium) is unexpectedly slow, developers cannot inspect the vendor library to see why — its internals are not source-available, and there are limited per-primitive configuration parameters to calibrate a simulator against. **(C2) No low-level control:** the vendor’s collective primitives (`xm.all_gather`, `xm.all_to_all`, ...) are called through a fixed API; there is no exposed schedule-level intermediate representation — which, in the open-stack systems MSCCL [12] and TACCL [50] target, is the surface that lets a user choose how chunks are split, routed through the network, or fused inside a collective.

This paper introduces a new design that addresses each of these challenges. **For (C1)**, we approximate visibility into the closed runtime by having an LLM agent build a cost-model simulator on the target deployment itself: the LLM

writes probing code that measures the simulator’s internal numbers against the actual device and model shape the workload will use. This does not require internal visibility into the runtime, but it gives a usable proxy for “is this strategy fast for this specific task?” whose estimates re-adapt to the deployment when either the device or the model shape changes — so the simulator stays workload-faithful where a hand-coded one would drift. **For (C2)**, we do not try to recover low-level control. We accept the vendor API as fixed and search one layer above it — over which primitives to call, in what order, and how to reshape, pad, or slice the input/output tensors around each call. The LLM proposes strategies at this layer; the workload-calibrated simulator scores them; top-scoring strategies are validated on real hardware before deployment.

One might ask: if the simulator’s own internal numbers come from on-device probes, isn’t this just the microbenchmark option in a different wrapper? Our ablation (§4.2) is set up to answer exactly that. The difference is in *what* we probe and *how* we probe it. Our simulator encodes hardware-specific performance models, and the LLM is guided to calibrate them using probe workloads that resemble real model-training settings (real tensor shapes, full step boundaries, multi-rank dispatch patterns), rather than the small isolated calls a standalone microbenchmark would measure. The probes therefore capture behavior that small microbenchmarks miss, while remaining far cheaper than running a full training harness end-to-end on each candidate strategy. When we ablate the simulator and let the LLM rank strategies by its own small microbenchmarks instead, the 1.40× OLMoE-10B speedup collapses to 1.00×: the LLM converges to structurally the same strategy practitioners already deploy, because microbench rankings are exactly what selected that strategy in the first place. In other words, the work-load-faithful simulator, not the LLM, is what picks the strategy that wins at actual training runs.

Closed-stack collective communication library. A growing class of production training accelerators operates on proprietary collective communication libraries. AWS Trainium [1, 2] — 30–40% better price-performance than NVIDIA H100 on transformer workloads and adopted at scale for Mixture-of-Expert (MoE) and Llama training — ships a Neuron Runtime that exposes collectives only through the PyTorch/XLA integration (XLA = Accelerated Linear Algebra, PyTorch’s compiler bridge to non-NVIDIA accelerators) — i.e. as `xm.*` entry points such as `xm.all_gather` — behind which the Neuron compiler emits NEFF binaries (Neuron Executable File Format; the compiled artifact loaded onto each NeuronCore) from unpublished sources. There is no user-configurable schedule layer: per-collective chunking and network-interface-card (NIC) routing cannot be reached from above the runtime. Google

TPU and Meta MTIA exhibit the same constraint: the programmable schedule surface that MSCCL- and TACCL-style synthesis depend on is not exposed. The schedule-synthesis paradigm of the open-stack era cannot be ported here in any straightforward way. Above the `xm.*` (or equivalent vendor) boundary, the design choice that remains is which primitive to call, in what order, and against what tensor layout. We refer to this as the *strategy* layer, in contrast to the *schedule* layer that MSCCL/TACCL operate on, which decides how each collective message is split into chunks, which network paths each chunk takes, and how the chunks are pipelined across devices. The strategy layer is less expressive but, as we show, contains strategies 1.4–3.2× faster than the developer baseline.

Related schedule-synthesis work and why it does not apply here. Automated synthesis of collective-communication algorithms has been studied at the *schedule* layer — which message chunks move along which paths, in what order, against which network adapters. MSCCL and MSCCLang [12] emit XML or DSL schedules for the NCCL/RCCL runtime [39] to consume; TACCL [50] uses a sketch language and an SMT solver; AutoCCL [64] adds tuning; SCCL [8] synthesizes Pareto-optimal algorithms from first principles. All share an infrastructural prerequisite: the underlying runtime exposes a programmable intermediate representation (IR) for the schedule.

Validation and contributions. We validate our design on the eight collective problems listed in Table 1 — four that arise in Mixture-of-Expert (MoE) training [52] and four that arise in dense-transformer (Llama-style) training — on a 7-node `trn1.32xlarge` cluster (224 ranks).

Problem	Role in the training step
<i>MoE-side</i>	
AllToAllV [29]	MoE token dispatch/combine; counts vary
Uniform AllToAll [69]	MoE dispatch/combine; counts equal
Ring KV [32]	rotate KV shards around the rank ring
Distributed CE [54]	loss over rank-sharded vocabulary
<i>Llama-side</i>	
PP cross-stage [19]	activations/grads between PP stages
TP MLP [54]	all-reduce after MLP in tensor parallel
FSDP	gather sharded weights per layer
prefetch [67]	
Layer-block AR [26]	all-reduce grads at layer blocks

Table 1. The eight collective problems we evaluate.

The baseline we measure against is the *internal-AWS-optimized strategy* AWS uses in production today to train large-scale MoE and dense transformer models on Trainium, built and maintained by five experienced AWS developers. On OLMoE-10B the deployed agent strategy delivers 1.40× steady-state step-time speedup with matched descent on a 2500-step real-OpenWebText [17] AdamW run; on Llama-7B it delivers 3.24×. Per-primitive per-step speedups range 4–12×, above the end-to-end ratio because constant non-collective compute dilutes the headline.

Deployment cost. A $1.40\times$ speedup on OLMoE-10B is worth roughly the following in a typical training deployment. A single 7-node trn1.32xlarge cluster runs at ~\$19/hour on-demand; a 14-day pretraining job at that scale is ~\$47k in compute alone. A $1.40\times$ end-to-end speedup on the steady-state step time cuts that to ~\$34k — ~\$13k saved per pretraining, or ~29% of the compute budget — with the same descent trajectory. The one-time cost of running the search is ~\$19 in Sonnet 4.5 LLM tokens and ~56 min of the 7-node cluster (Table 10). At the scale of production MoE and Llama training the search cost amortizes across a single pretraining and every fine-tune that reuses the deployed runtime module. The Llama-7B $3.24\times$ number sits at the top of this range because per-microbatch dispatch overhead (M dispatches per step in the baseline, one in the bundled agent) is the ratio-limiting term.

The contributions are (i) to our knowledge the first system that finds faster collective-communication strategies on top of a closed-source vendor library, delivering $1.40\times$ end-to-end on OLMoE-10B and $3.24\times$ on Llama-7B over the production baseline; (ii) a cost-model simulator whose per-term constants are re-fit per-cluster and per-model-shape by LLM-written on-device probes, keeping the simulator workload-faithful without hand-coding new probe scripts for each deployment; and (iii) the closed-stack search loop combining the above with LLM-driven mutation and hardware validation, with code emitted as drop-in runtime modules and algorithm-equivalent descent verified on real to-kens.

Scope and generalization. We instantiate our design on AWS Trainium. The six cost-model terms are Trainium-specific, but the categories they fall into (per-graph-launch overhead, per-collective bandwidth ceiling, device-vs.-host copy cost, runtime cache-reload cost) are general to closed-source vendor runtimes that compile their own collective binaries. Google TPU and Meta MTIA each expose closed equivalents; we expect the strategy-search design to port with the cost-model term set re-calibrated, and leave that to follow-up work.

2 System Model

2.1 The search loop

Notation. A collective problem p has a Python reference $\text{ref}_p(\cdot)$, a seed library $\mathcal{S}_p$ of human-written strategies, and fixed I/O shapes. The search space Σ_p is the set of functions with p 's signature that compose primitives from the vendor set $\mathcal{V} = \{\text{all_gather}, \text{reduce_scatter}, \text{all_reduce}, \dots\}$ together with local host-side ops the accelerator's compiler lowers to device code. A candidate $s \in \Sigma_p$ is *admissible* if $\text{CORRECT}(s) = 1$ (matches ref_p at $\text{ws} \in \{4, 8\}$) and $\text{GATE}(s) = 1$ (compiles and runs at the training-shape configuration). Phase 1 fits a cost model $\text{Sim} : \Sigma_p \rightarrow \mathbb{R}_{\geq 0}$ (Section 2.2); Phase 4 evaluates GATE. The deployed strategy is

$s_p^\star = \arg \min_{s \in \Sigma_p^\dagger} \text{Sim}(s)$ over admissible $\Sigma_p^\dagger$. The LLM $\mathcal{M}$ is a stochastic proposer $\mathcal{M} : \Sigma_p \rightarrow \Sigma_p$; Phase 3 instantiates it as one of three search shapes (strategy-enumerate / cc-react / multi-island) under a bounded $\mathcal{M}$ -call budget. Figure 1 illustrates the five phases.

Phase 1: agent-driven on-device calibration. We refer to the LLM-driven search loop as the *agent*. In Phase 1, the agent is given a set of measurement *tools* (Table 2) and designs the probe campaign autonomously: tensor sizes, primitives, repetitions, and sanity checks (e.g. varying tensor contiguity to isolate hidden memory-copy overhead). Each tool returns an empirical scalar or function that wires into one term of Eq. 1. We supply the tools, not the experimental design, so the LLM cannot fabricate numbers but can exploit domain knowledge to choose informative probes. Two terms in the tool list: *EFA* (Elastic Fabric Adapter) is AWS's RDMA-based cross-node network; *amortization* is the empirical drop in per-call cost when k collectives fire consecutively, because the runtime overlaps per-call overhead.

Tool	What it measures
m_collective_lat	per-prim latency vs. bytes / world size
m_p2p_transfer	cross-node point-to-point bandwidth over EFA
m_xla_op_overhead	per-op floors for local XLA ops
m_compilation_cost	cold / cache / reload NEFF compile times
m_launch_overhead	per-graph-dispatch (mark_step) tax
m_memcpy_thru	seq. and strided host/device copy bw
m_b2b_amortization	amortization curve for k consecutive collectives

Table 2. Phase-1 measurement tools the agent is given (prefix $m_ = \text{measure_}$).

Phase 2: Baseline evaluation. A per-problem seed library is scored through the simulator. It always contains the *internal-AWS-optimized baseline* for that primitive plus a few naive alternatives. The baseline is the strategy AWS uses in production today for large-scale MoE and dense-transformer Trainium training, built and confirmed by five experienced AWS developers — the strategy practitioners deploy, not a strawman. The LLM is not told which seed is the baseline, which both calibrates simulator ordering against established practice and seeds Phase 3.

Phase 3: Agent-driven search. Strategy-enumerate: the LLM enumerates K structurally distinct sketches (“pack and gather”, “per-source slice loop”, “masked all-reduce over stacked microbatches”, $\dots$), implements each independently, then refines the top-2 by simulator score under a bounded budget (§2.4).

Phase 4: Hardware validation. Each Phase-3 survivor runs a 20-iteration HW microbench plus a 10-step training-validation (TV) harness on a synthetic 8-layer LM. Crashes, OOMs, NaN at world-scale, or shape-gate failures exclude

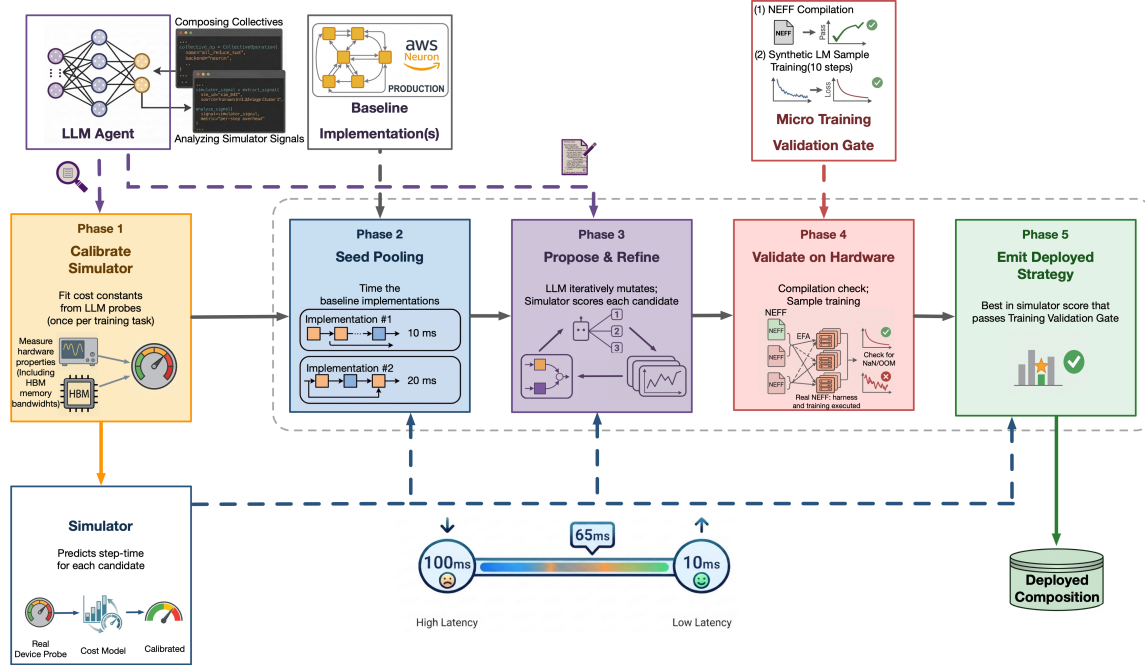


Figure 1. Closed-stack search loop.

the candidate regardless of simulator score; failed candidates get LLM-assisted recovery (up to two attempts). HW timing itself is not the ranking signal.

Phase 5: Code generation. The lowest simulator-score survivor among HW+TV passers wins. HW microbench isn't the ranking signal because it measures isolated-call latency with cached NEFFs and misses the NEFF-cache-eviction cost that appears in real training.

2.2 Cost model

The simulator builds a per-step time estimate as the sum of six physically-grounded terms:

$$T_{\text{step}} = \underbrace{\sum_{i \in \mathcal{L}} c(\text{op}_i, b_i)}_{T_{\text{local}}} + \underbrace{\sum_j \max(d_{\text{amort}}^{(j)}, b_j/w_{\text{eff}})}_{T_{\text{bw}}} + \underbrace{d_{\text{full}} + (k-1)d_{\text{amort}}}_{T_{\text{coll}}} + \underbrace{2\ell m}_{T_{\text{launch}}} + \underbrace{C(b_{\text{max}})r/S}_{T_{\text{NEFF}}} + \underbrace{B/w + \pi_{\text{net}}}_{T_{\text{net}}} \quad (1)$$

2.2.1 Per-op local cost (T_{local}). T_{local} sums per-op cost over $\mathcal{L}$, the chronological list of local XLA ops recorded at trace time. View-only and fused-elementwise ops pay a fixed floor; cat/stack are bytes-aware at $\max(\text{floor}, b/w_{\text{seq}})$; contiguity-dependent ops (reshape/contiguous, index_select, host-side torch.tensor) pay $\max(\text{floor}, b/w_{\text{strided}})$ on non-contiguous sources, blocking

Algorithm 1 End-to-end closed-stack collective-strategy search.

Require: $P, \text{ref}(\cdot), S_P, \text{LLM } \mathcal{M}, \text{profiling tools } \mathcal{T}, \text{hardware } H.$

Ensure: deployable runtime module for problem P .

```

1:  $\Theta \leftarrow \mathcal{M}.\text{CALIBRATE}(\mathcal{T}, H)$  ▷ Phase 1
2:  $\text{Sim}(\cdot) \leftarrow \text{BUILDCOST}(\Theta)$  ▷ Eq. 1
3: for  $s \in S_P$  do ▷ Phase 2
4:    $\text{score}(s) \leftarrow \text{Sim}(s); \text{VERIFY}(s, \text{ref}, \text{ws} \in \{4, 8\})$  ▷
   verify on small rank counts  $\text{ws} \in \{4, 8\}$ 
5: end for
6:  $C \leftarrow \text{INITPOOL}(S_P)$  ▷ Phase 3 (strategy-enumerate)
7: for  $r = 1 \dots R$  do
8:    $s' \leftarrow \mathcal{M}.\text{PROPOSE}(C, \Theta)$ 
9:   if  $\neg \text{VERIFY}(s', \text{ref})$  then continue
10:  end if
11:   $\text{score}(s') \leftarrow \text{Sim}(s'); C \leftarrow C \cup \{s'\}.$ 
12: end for
13: for each Phase-3 survivor  $s$  do ▷ Phase 4
14:    $t_{\text{hw}}(s) \leftarrow \text{MICROBENCH}(s, H)$ 
15:    $\text{SHAPEGATE}(s, H); \text{TVGATE}(s, H).$ 
16:    $s \leftarrow \mathcal{M}.\text{RECOVER}(s)$  if gate fails (up to  $2\times$ ).
17: end for
18:  $s^* \leftarrow \arg \min_{s \in \text{survived}} \text{Sim}(s)$  ▷ Phase 5
19:  $\text{EMIT}(\text{runtime module for } P, s^*); \text{deploy to } H.$ 

```

a narrow→reshape→contiguous chain that moves data 248

at strided bandwidth. Remaining ops pay their measured isolated cost.

2.2.2 Collective bandwidth and back-to-back amortization (T_{bw}, T_{coll}). T_{bw} floors each dispatch at b_j/w_{eff} , with w_{eff} from the Phase-1 topology object (cross-node: per-EFA bandwidth $\times$ adapters/node; intra-node: per-NeuronLink, Trainium’s on-package interconnect). Nothing is hardcoded per problem. T_{coll} charges k back-to-back dispatches with an empirically fit amortization schedule: $k=1$ pays full per-call, moderate chains pay a pipeline-fill discount, deeper chains pay a further discount as XLA fuses adjacent collectives into one NEFF segment. Discounts are auto-fit from `measure_back_to_back_amortization`.

2.2.3 Graph-launch and NEFF reload ($T_{launch}, T_{NEFF}, T_{net}$). T_{launch} pays ℓ twice per `autograd.Function` boundary (forward + implicit backward `mark_step`); an AST-derived outer-loop tax contributes to m , while inner per-tensor loops don’t (XLA fuses them). T_{NEFF} amortizes reload cost $C(b_{max})$ over S steps with r reload events; T_{net} adds the static-byte transfer term.

2.2.4 Reward-hacking-resistant structural terms. Three structural terms catch HW failure modes that pure-cost ranking misses.

1. A **fusion-credit** pass discounts each local compute op adjacent to a collective with no stack/cat barrier between them, capturing the compiler-level fusion of element-wise/reduction ops into a collective’s segment.
2. An **HBM safety** term applies a smooth quadratic penalty when estimated peak exceeds the per-NeuronCore HBM budget (16 GB on `trn1`). An AST pass recognizes explicit byte-budget assignments (e.g. `chunk_size_bytes = ...`) and propagates the cap so a bucketed algorithm isn’t charged for the flat tensor it would otherwise appear to materialize.
3. **Primitive-viability** terms give $+\infty$ cost to two real HW failure modes: `xm.collective_permute` rings at large world sizes (compiler aborts) and `xm.all_reduce` payloads above the per-collective size limit.

2.2.5 Phase-5 ranking: simulator, not raw microbench. Phase 5 ranks by simulator score; HW microbench and TV are correctness gates only. Two reasons. First, HW/TV measurements often fail to generalize to real training: the microbench runs at small world size with cached NEFFs and the TV harness runs a tiny 8-layer LM that misses the NEFF-reload and back-to-back amortization patterns that dominate at 224 ranks — the cross-scope inversion this paper documents. Second, raw wall-clock is a sparse reward an LLM can game (stacking metadata views that fuse for free in isolation but force an extra `mark_step` in training); the simulator’s per-term decomposition with

tool-measured floors stops that. Fallback: when no candidate clears the gates, deploy the lowest-simulator-score baseline.

2.3 On-device profiling (Phase 1): the agent builds its own simulator

The cost model is not a fitted regression: each parameter $\Theta = (\Theta_{net}, \Theta_{launch}, \Theta_{NEFF}, \Theta_{copy}, \Theta_{neff-cache})$ comes from the agent’s probe tools (Table 2): $\Theta_{net} = T(b, N) = \alpha(N) + \beta(N) b$ per primitive; Θ_{launch} is per-graph-launch overhead (50–200 μs on Trainium); Θ_{NEFF} is compile/load cost at four cache tiers, amortized over $K_{amort} \approx 5000$ NEFF-resident steps; Θ_{copy} is sequential/strided memcopy bandwidth; $\Theta_{neff-cache}$ is the back-to-back amortization curve. Phase 1 takes 8–12 wall-minutes on the 7-node cluster, once per (hardware, software-version) tuple. Hiding Θ collapses OLMoE-10B from $1.40\times$ to $1.00\times$ (\$4.2).

2.4 Strategy-enumerate loop (Phase 3)

Phase 3 runs a bounded LLM-evolution loop on the Phase-2 seed pool S_P . We use an LLM (rather than random sampling, simulated annealing, or a fixed-grammar GA) because the task is to *author* deployable code; recent LLM-driven code-synthesis work is state-of-the-art over rule-, mutation-, and random-search baselines for compute and communication kernels [18, 38, 47]. Strategy-enumerate is two-stage.

Stage 1: K -way enumeration. One LLM call emits K structurally distinct sketches $\{\sigma_1, \dots, \sigma_K\}$ ($K=5$) given the problem signature, S_P implementations with their Sim scores, and Θ flattened per-op. Each σ_k is implemented by a second LLM call, sandbox-tested against ref at $ws \in \{4, 8\}$ in fp32, and scored by Sim. Candidates passing correctness and bf16 robustness pass on.

Stage 2: bounded refinement of the top-2. The top two refine for $R=2$ rounds each: identify the dominant cost term $\tau^* \leftarrow \arg \max_{\tau} \tau(s)$, prompt with $(s, \text{Sim}(s), \tau^*, \text{breakdown}(s), \text{history})$ for s' , accept if $\text{Sim}(s') < \text{Sim}(s)$ and s' passes the sandbox. Section 4.1 compares against a single-trajectory ReAct agent and a multi-island GA.

Cost calculus vs. end-to-end trials. Table 3 quantifies the loop’s advantage over end-to-end candidate trials: $O(1)$ hardware validation per problem regardless of K , versus $O(K)$ cluster hours for e2e.

3 Evaluation

Setup (shared). $7 \times \text{trn1.32xlarge}$ in one AWS VPC with EFA networking (32 NeuronCores/node, 224 ranks total, 8 EFA adapters/node at 12.5 GB/s). All measurements use `NEURON_NUM_RECENT_MODELS_TO_KEEP=1` — the practitioner default for large-LLM Trainium training — so

	E2E trials	Our loop
Score one candidate	full training run (~1 h)	sim. query (~ms)
HW runs / problem	every candidate	sim. winner only
Phase-1 calibration	N/A	~30 min cluster
LLM cost / problem	0	~\$2.40
Scaling in K	$O(K)$ cluster h	$O(1)$ cluster h
<i>8 problems, $K=5$ per problem, 7-node trn1.32xlarge:</i>		
Cluster time	~40 h	~8 h
Cluster cost (\$150/h)	~\$6,000	~\$1,200+\$19 LLM

Table 3. Cost calculus, 8-problem suite. The $\sim 5\times$ saving at $K=5$ is a lower bound: strategy-enumerate also scores simulator-refined variants for free.

Problem	1n bench (ms/call)	7n bench (ms/call)	7n train (ms/step)
	base OVERLAYCCL	base OVERLAYCCL	base OVERLAYCCL
<i>OLMoE collectives</i>			
AllToAllV	1.83	1.95 1.89	3.04 4410 3352
Uniform A2A	46.95	47.87 55.4	52.3 373.2 206.2
Ring KV	3.57	1.07 2.95	2.02 13.49 10.32
dxe (dist. CE)	1.64	0.37 1.87	0.57 16.74 14.63
<i>Llama primitives</i>			
PP cross-stage	1.78	2.38 1.34	2.37 21.36 4.11
TP MLP	1.91	2.83 0.64	1.67 8.04 1.93
FSDP prefetch	1.21	2.00 1.37	1.39 8.52 2.01
Layer-block AR	0.89	2.94 1.15	4.35 9.40 2.26

Table 5. Per-primitive latency at three scopes: 1-node 32-rank bench, 7-node 224-rank bench, 7-node per-step training. Bold marks the faster in each pair. AllToAllV dominates the OLMoE step, so its 7n-train column tracks Table 6.

Harness	Backend	Wall (min)	Steady (ms)	Final loss	Speedup
<i>OLMoE-10B</i>	baseline	183.6	4404.3	6.843	1.00×
	OVERLAYCCL	130.2	3139.8	6.945	1.40×
<i>Llama-7B</i>	baseline	2.5	71.3	4.970	1.00×
	OVERLAYCCL	5.9	22.0	4.970	3.24×

Table 6. End-to-end speedups over the developer baseline (algorithm-equivalent descent; setup in Table 4).

	Per-prob. mb	OLMoE-10B e2e	Llama-7B e2e
Scale, steps	1n·32r, 5k	7n·224r, 2.5k	7n·224r, 300
Model shell	DeepSeek-MoE-Lite	OLMoE	Llama-2-7B
LYRS/DM/HD	12/2048/16	8/2048/16	32/4096/32
NEXP/TOPK/XD	64/6/1408	224/8/1024	—
S/B/V	256/1/32768	256/1/32256 ^s	1024/1/32000 ^s
Routing	token-choice	expert-choice	dense
Collective(s)	top-K one per script	AllToAllV, dxe	PP, TP MLP, FSDP, lbar

Table 4. Evaluation harnesses (bf16, full autograd). ^ssharded vocab.

baseline compute) — well below any real pretraining, so the 300-step harness undersells end-to-end wall time.

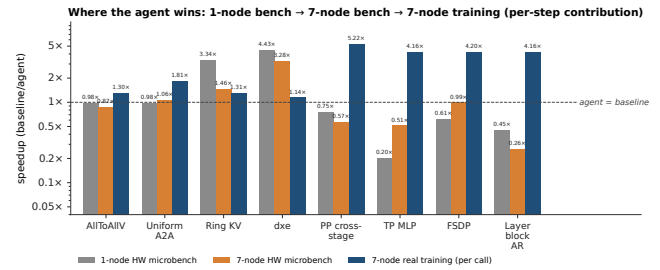


Figure 2. Cross-scope inversion. Per-problem speedup (baseline/agent) at 1-node bench, 7-node bench, and 7-node training scope; bars are strategy-enumerate numbers from Table 5, dashed line marks agent=baseline. Most primitives’ per-call ranking at bench scope inverts at training scope.

3.2 OLMoE end-to-end training

The OLMoE harness matches OLMoE [35]: multi-head attention with RoPE [55] and RMSNorm [66], a MoE block with SwiGLU experts [51], and expert-choice routing [69] (fixed per-expert receive counts, no host-side book-keeping). Two collectives are toggled independently: AllToAllV around the MoE block (twice per layer) and distributed cross-entropy (dxe) over the rank-sharded vocabulary. **For AllToAllV**, the agent deploys pack-and-gather (see the case study in §3.5). **For dxe**, the NeuronX-Distributed and Megatron-LM recipes [4, 54] compute

the loss as an All-Gather (AG) of per-shard logits then `F.cross_entropy`, because past Neuron-SDK versions silently returned NaN or biased loss on rank-sharded reductions. The agent’s HW+TV gate at our SDK (neuronx-cc 2.23, torch-neuronx 2.8) verifies bf16 is reliable here, so the agent replaces the AG+loss form with a two-AllReduce (AR) trick over pre-reduced (max, logsumexp) statistics — no logit tensor gathered, the $(B \cdot S, V)$ intermediate goes away. The *OLMoE-10B* deployment is ~ 11.5 B parameters across 224 ranks. On a 2500-step real-OpenWebText run (Table 6) both backends descend from $\log V \approx 10.4$ to ≈ 6.9 , agreeing within ± 0.15 at every 50-step checkpoint. Table 7 sweeps sequence length S (SEQLEN), model dimension DM , and layer count; speedup stays in a tight $1.39\text{--}1.41\times$ band, so the headline is configuration-robust.

3.3 Llama training

Dense transformer training differs structurally from MoE: each microbatch’s collectives sit in their own compiled graph and Neuron cannot fuse collectives across microbatches, so a step with M microbatches ($M=N_{MB}$ elsewhere) issues M separate calls per collective. The four Llama-side problems (PP cross-stage send/recv, TP MLP AR, FSDP weight prefetch, layer-block AR; Table 1) are all issued in this per-microbatch form by the production AWS recipe [3, 4]. Practitioners prefer it because (i) a naive mega-graph unrolling forward+backward across all M microbatches exceeds Trainium’s memory/compile budget while one graph per microbatch fits, and (ii) each microbatch’s small collective looks cheap under per-call microbench. The bundled alternative (defined below, distinct from the mega-graph) keeps the per-microbatch compute graph intact and only merges the collective calls: M per-microbatch AllReduces or AllGathers over gradients/activations become one call over an $M\times$ -larger tensor per step. Dispatches drop from M to 1 without enlarging the compute-graph tensor footprint, so the mega-graph OOM signal never fires.

Why isn’t collective bundling just the obvious hand fix? Dispatch bundling and gradient bucketing are widespread on open GPU stacks (NCCL/CUDA/A100), so the natural objection is that Llama-side speedups reduce to a known optimization. That premise doesn’t survive contact with a closed vendor stack: Neuron is closed-source, its NEFF format has quirks and per-shape compile failures GPU folklore doesn’t cover, and practitioners rely on microbench-driven intuition. Two facts push back. First, the microbench does not favor bundling: the $M\times$ -larger single-collective tensor enlarges the NEFF and shifts work into the compile-cost term, showing up as a per-call regression (Table 5: PP send/recv $1.34 \rightarrow 2.37$ ms, TP MLP $0.64 \rightarrow 1.67$ ms), and a shape-inference edge case further discourages hand exploration. Neither binds at training scope, where the launch tax paid M times

per step dominates the compile-cost delta (which under `NEURON_NUM_RECENT_MODELS_TO_KEEP=1` amortizes to a rounding error). The simulator combines a compile-cost term (largest single-collective tensor) against a launch-tax term linear in M ; under Trainium’s calibrated constants the launch tax wins by M and the ranking flips.

Second, and more strongly, Phase 4 matters even when the simulator ranks a bundled strategy first. **FSDP weight-prefetch** is the interesting case: the stack-the- M -shards-and-issue-one-AG pattern that wins PP and TP MLP *doesn’t* survive Phase 4 at 224 ranks — the packed $(M \cdot N_{layers}, ws, shard_size)$ tensor exceeds the per-rank HBM budget the compiler tolerates when materialized in one graph, and compile aborts. The simulator ranks it best and the LLM enumerates it confidently, but Phase 4 catches it and the loop falls back to the next-best Phase-3 candidate. That fallback — the deployed FSDP strategy — is a per-microbatch per-layer loop of $M \cdot N_{layers}$ AGs, structurally identical to the developer baseline at the call level. The agent’s contribution here is not dispatch collapsing but lifting the per-microbatch `mark_step` the developer keeps between microbatches, letting NeuronCC fuse all $M \cdot N_{layers}$ AGs into one HLO graph per step. Table 5’s FSDP row ($8.52 \rightarrow 2.01$ ms) is that graph-count drop, not a dispatch drop. The pipeline’s contribution is thus not *inventing* bundling: it is *surfacing* that Trainium-side per-call regressions don’t bind at training scope, and *robustly falling back* to a deployable next-best when a theoretically-best candidate hits a hardware constraint the LLM could not predict from closed vendor documentation. The four Llama-side problems look like different parallelism patterns (pipeline, tensor, data) but reduce to one strategy template once the per-microbatch `mark_step` structure is made explicit — a regularity the simulator’s amortization model (§2.2) surfaces without running on hardware.

We use two harnesses: a Llama-block sweep at four shapes (amp1–amp4, Table 7) and a Llama-7B end-to-end run under the same toggle. Per-call wins are modest ($\sim 1.04\text{--}2\times$) but multiply into $4\text{--}6\times$ per-step ratios. *M scales the speedup, not the ranking*: the per-microbatch `mark_step` decouples compute and comm, so the headline tracks M ($\sim 3.49\text{--}3.62\times$ at $M=4$, $\sim 5.84\text{--}6.48\times$ at $M=8$; amp2 is the default in Table 6).

Llama-7B end-to-end. At a Llama-7B shape, 200 warmup-dropped steps at 224 ranks yield $3.24\times$ steady-state (Table 6), with bit-identical per-microbatch-normalized loss (4.970 vs 4.970) on random tokens — an algorithm-equivalence proof for the swap.

3.4 Multi-seed retest

We re-ran each cell of Tables 5 and 6 at $n \geq 5$ seeds. Llama-side primitives use a per-call probe (median $\times$ per-step count: N_{MB} for baseline, 1 for OVERLAYCCL); Ring KV and

Harness	Config	base (ms/step)	OVERLAYCCL (ms/step)	Speedup
OLMoE-10B	SEQLEN=128	2696.9	1943.1	1.39×
	SEQLEN=256 (default)	4404.3	3139.8	1.40×
	SEQLEN=512, DM=1024	4286.1	3076.4	1.39×
	LAYERS=4	2245.8	1593.0	1.41×
Llama-block	amp1 (S=1024, M=4)	46.26	12.68	3.65×
	amp2 (S=2048, M=4)	60.08	17.14	3.51×
	amp3 (S=1024, M=8)	90.62	15.43	5.87×
	amp4 (S=2048, M=8)	112.49	17.95	6.27×

Table 7. Configuration-knob sweep (ms/step, 1000-step steady mean). OLMoE is configuration-robust at 1.39–1.41×; Llama-block tracks M because the bundled agent collapses M per-microbatch dispatches into one.

and older NEFFs get evicted, so AG+RS’s two-collective path yields a larger NEFF with more reloads; and (2) the compile-cost term scales super-linearly with the largest single-collective tensor, and AG+RS’s $O(N \cdot N \cdot c_{\max})$ intermediate is $N \times$ pack-and-gather’s send buffer. Neither fires in the microbench; both fire at training scope. Phase 5 emits pack-and-gather — $\approx 25\%$ of the 1.40× OLMoE speedup.

3.6 Cluster-size generalization: 7-node to 3-node

The deployed strategies bake in *topology constants* (total ranks, half, nodes) at codegen — (224, 112, 7) on 7 nodes. Redeploying at (96, 48, 3) and (160, 80, 5) (re-deriving model-shape parameters so the model fits), Table 9 shows the speedup persists in both directions — not a 7-node-shape overfit.

Harness	base (ms)	OVERLAYCCL (ms)	Speedup
OLMoE-10B 7n (224r)	4404.3	3139.8	1.40×
OLMoE-8B 5n (160r)	3258.0	2173.0	1.50×
OLMoE-5B 3n (96r)	2547.4	1422.0	1.79×
Llama-7B 7n (224r)	71.30	22.00	3.24×
Llama-7B 5n (160r)	67.94	24.77	2.74×
Llama-7B 3n (96r)	67.82	27.20	2.49×

Table 9. Cluster-size generalization: same runtimes re-deployed at 5-node and 3-node with topology constants re-fitted. OLMoE speedup widens as cluster shrinks; Llama speedup compresses.

Primitive	n	baseline (ms/step)	OVERLAYCCL (ms/step)	Speedup
<i>Llama-side</i>				
PP send/rcv	5	13.22 ± 0.14	1.16 ± 0.22	11.41×
FSDP prefetch	5	2.03 ± 0.68	0.45 ± 0.14	4.54×
TP MLP	5	1.51 ± 0.69	0.37 ± 0.15	4.02×
vocab dxe	5	2.92 ± 0.31	0.88 ± 0.10	3.30×
Grad AR	5	1.00 ± 0.10	0.79 ± 0.21	1.27×
<i>OLMoE-side</i>				
Ring KV	5	10.51 ± 0.19	6.73 ± 0.12	1.56×
dxe	5	15.59 ± 0.09	10.47 ± 0.38	1.49×
Uniform A2A	5	3319.9 ± 319.6	2358.1 ± 201.8	1.41×
<i>Llama-block harness (amp3 shape)</i>				
Llama-block	5	27.97 ± 0.16	3.70 ± 0.28	7.56×

Table 8. Multi-seed retest at $n \geq 5$ seeds; every row is a clean $2\sigma_{\bar{x}}$ OVERLAYCCL win. The Llama-block row is the amp3 harness, *not* the Llama-7B headline (Table 6).

3.5 Case study: AG+RS vs pack-and-gather

xm. all_to_all_v fails to compile at multi-node world sizes on Neuron, so practitioners synthesize AllToAllV. The developer SOTA is AG+RS: pad to $c_{\max}$, AG the global buffer, permute+reshape, then RS. RS *here is not arithmetic reduction*: after AG every rank holds the same buffer, so summing N identical copies and scaling by $1/N$ is a no-op — RS is a routing primitive, the only vendor call that natively returns per-rank shards in one dispatch with a predictable NEFF. The alternative, extracting shards by host-side slicing, looks worse: a for src in range(N) loop of $N=224$ slice+cat ops expands into $O(N)$ trace nodes the developer cannot assume will fuse, and the 20-iter microbench confirms the fear (1.66 vs 2.75 ms/call). The agent nevertheless proposes **Pack-and-gather**: pack variable-length data into a fixed-size send buffer with rank i ’s payload at $i \cdot c_{\max}$; one AG; extract each rank’s slot via a contiguous 1D slice (gathered[base:base+count]) and cat per-source slots. At training scale (1) each step compiles a fresh HLO

4 Ablation Study

The main results use the default *strategy-enumerate* Phase 3 loop. Two ablations show that (i) alternative Phase 3 styles reach the same performance at $\sim 25\%$ higher search cost, and (ii) stripping the simulator’s cost feedback collapses the 1.40× OLMoE headline to 1.00×.

4.1 Alternative Phase-3 loops: cc-react and multi-island GA

cc-react is a single-trajectory ReAct agent on Claude Code [7] maintaining a champion scratchpad plus per-op cost feedback. **Multi-island GA** runs three-plus populations with LLM-mutation and occasional champion exchange, modeled on AlphaEvolve [38]. Both share Phase 1 and Phase-4/5 gates.

Result: per-problem cells within $\sim 10\%$ of strategy-enumerate; OLMoE end-to-end reproduces 1.40× within $< 0.5\%$; Llama amp1-amp4 within $\sim 10\%$. All three converge to the same deployed winner.

Why strategy-enumerate is preferred: it reaches that strategy $\sim 25\%$ faster and $\sim 20\%$ cheaper on LLM spend (Table 10) under a common 10-call/problem budget: one enumeration call for $K=5$ sketches, K implementations, $2R=4$ refinements. Six of ten calls *explore*, four refine — the

greedy-vs-portfolio trade-off [25, 48]: one round on K templates covers more ground than repeated refinement of one.

Phase-3 style	Wall (min)	LLM cost (Sonnet 4.5)
strategy-enumerate (default)	56	\$19
cc-react	75	\$22
multi-island	78	\$24

Table 10. Per-style search cost over the eight collective problems on the 7-node cluster.

4.2 No-simulator ablation: necessity of Phase-3 cost feedback

To isolate simulator feedback, we hide Sim’s output from Phase 3 (Phase 1 still builds it; the saved refinement budget lets the agent write its own microbenchmarks); Phase 5 picks the winner (“LLM-judge”) from Phase-4-gate survivors by HW microbench latency. Search wall-time is unchanged (~59 vs ~56 min). The LLM-judge falls back to whatever wins the microbench — structurally the inline developer baseline for all three load-bearing OLMoE primitives — and the end-to-end ratio collapses to $1.00\times$ (Table 11).

Harness	base (ms)	no-sim (ms)	Ratio
OLMoE-10B (a2av + dxe)	4399.8	4393.4	$1.00\times$
Llama-7B [◊]	76.9	76.0	$1.01\times$

Table 11. No-simulator ablation end-to-end. OLMoE $1.00\times$ is the central finding: stripping simulator scores collapses the $1.40\times$ headline. [◊]Llama-7B drops PP, uses TP+FSDP+lbar.

5 Portability to other closed-stack accelerators

Does the design transfer to other closed-stack accelerators (Google TPU, Meta MTIA) or exploit a Trainium specific? Every platform input is either replaceable or platform-agnostic: **Phase 1** (calibration) times a single xm.* -like primitive at a chosen shape — `torch_neuronx+torch_xla` on Trainium, `jax.pmap` or `torch_xla/tpu` on TPU, internal runtime on MTIA. **Phase 2** (seed pool) needs correct seeds; ours (`allgather+slice`, `reduce_scatter+pack`, `collective_permute-ring`) are expressible in JAX and MTIA, which expose the same primitives. **Phase 3** needs no platform-specific input beyond the calibrated simulator and seed pool. **Phase 4** needs a launcher — `torchrun` on Trainium, `xla_multiprocessing.spawn` or JAX multi-controller on TPU. Nothing hardcodes Trainium bandwidth, NeuronLink topology, or EFA latency; those are Phase-1 measurements re-taken per deployment. Porting replaces

Phase-1’s timing wrapper and Phase-4’s launcher; proposer and simulator are unchanged.

Cost-model constant swap. Substituting the six Trainium constants with published TPU v5p equivalents (ICI 600 GB/s, DCN 25 GB/s/port, XLA compile 350 μs , etc.) and rescoring the seed pool: Spearman rank correlation across nine templates is $\rho=1.00$; top-3 (`fused_alltoall`, `hierarchical`, `multinode_hierarchical`) and bottom-3 (`allgather_reduce_scatter`, `allgather_slice`, `hybrid_ag_perm`) survive intact; absolute scores drop 40–55% (tracking $\sim 3\times$ higher ICI bandwidth) but the ranking Phase 3 acts on is unchanged. The structural design is portable; only calibration is platform-specific. TPU cluster runs are future work. Out-of-scope: NCCL-supported GPU clusters, where NCCL (with MSCCL [12], TACCL [50], AutoCCL [64]) already exposes the schedule-level IR our (C2) constraint assumes away — schedule synthesis applies there.

6 Related Work

Distributed training systems above the collective layer. Systems such as ZeRO [42, 45], DeepSpeed [44], Megatron-LM [26, 37, 54], PyTorch DDP and FSDP [30, 67], GPipe [19], PipeDream [36], Horovod [49], BytePS [24], and FlexFlow [22] optimize a different layer than this paper does: they decide *how the training job itself is laid out across devices* — how model weights, optimizer states, activations, and microbatches are partitioned, replicated, or pipelined — and then leave the collective calls themselves to the underlying runtime (NVIDIA’s collective-communication library NCCL [39] or AMD’s equivalent RCCL on open stacks, or to a schedule-synthesis framework like MSCCL [12] or TACCL [50] where that is available). Our search instead fixes the model-parallel layout and optimizes the next layer down: *how each individual collective is implemented* — which vendor primitive to call, in what order, and how to reshape, pad, or slice the input/output tensors around each call — against the closed Trainium API where xm.* primitives cannot be modified or introspected. Strategy-layer systems do not catch vendor-specific pathologies on this stack (e.g., `xm.all_to_all` fails to compile across multiple nodes on Trainium; a tensor reshape that looks free on paper actually forces the runtime to physically copy the full tensor at slower, non-sequential memory bandwidth), and schedule-synthesis frameworks rely on rewriting schedules *below* the runtime boundary, which Neuron disallows. We treat xm.* as fixed and search strategies *above* it.

Cost-model simulators for distributed training. SimAI [61], ASTRA-sim [43], and ASTRA-sim 2.0 [63] score training-system choices against a cost-model simulator instead of running them end-to-end. They are written and tuned against a fixed reference configuration; we discuss

the resulting drift problem and our LLM-recalibrated alternative in §1 and §2.2.

Collective synthesis below an open boundary. A line of work synthesizes collective algorithms by exposing the in-fabric schedule: SCCL [8] casts it as a constraint problem; MSCCL/MSCCLang [12] compiles a DSL to an NCCL schedule IR; TACCL [50] adds communication sketches and topology-aware search; Blink [60] targets multi-GPU servers; CoCoNet [20] jointly optimizes compute+communication; AutoCCL [64] auto-tunes low-level NCCL parameters under compute-comm interference. All depend on a vendor library that allows replacing or parameterizing the in-fabric schedule, a prerequisite met on open NVIDIA/AMD stacks but not on closed-stack accelerators such as AWS Trainium (Neuron Runtime), Google TPU (where collectives are emitted by the XLA compiler as nodes in its compute-graph IR, called HLO — High-Level Operations), and Meta MTIA — where the in-fabric schedule lives behind a vendor compiler and is not externally addressable. The Neuron stack we use as our case study has no analogue of MSCCL-IR; we therefore search above the primitives, not beneath them. The same closed-stack constraint holds on TPU and MTIA, which makes the strategy-search paradigm we describe potentially applicable to them as well (§5).

Algorithm discovery with LLMs. FunSearch [47] pioneered the LLM-as-mutator loop for mathematical discovery; AlphaEvolve [38] extended it to code-level optimization; ShinkaEvolve [27] emphasized sample-efficient open-ended evolution. AlphaTensor [15] and AlphaDev [34] use deep RL for matmul/sorting discovery. LLM code-generation work (Codex [9], AlphaCode [31], Self-Refine [33], Reflexion [53], ToT [65], Self-Instruct [62]) further established LLMs as practical mutation oracles given a feedback loop. All search the *algorithm* or *program text* space; in our setting the same strategy admits many syntactic forms and the cost surface is dominated by hardware quirks (NEFF cache, EFA pipelining, implicit copies) none of these frameworks model. We take the LLM-mutation loop structure from this lineage and graft it onto a closed-stack-appropriate cost function; the headline configuration is a single-trajectory ReAct agent because in our setting the cost model and profiling toolkit already do the work island parallelism would otherwise do — and the single-trajectory variant is cheaper. Multi-island appears as an ablation.

Kernel autotuning and graph optimization. TVM [10], AutoTVM [11], Ansor [68] search kernel-implementation choices; Halide [40] and Triton [56] provide DSLs; Tensor Comprehensions [59], MLIR [28], TASO [21] operate at the graph level. LLM-driven kernel-optimization systems (STARK [5], K-Search [6], Astra [13], KernelEvolv [46]) refine individual GPU kernels by 1.25–17×. Our

problem is one level above: the Neuron compiler writes each `xm.*` kernel; we vary which primitives are composed and how, not their internal schedule.

MoE and Llama communication patterns. The Sparsely-Gated MoE layer [52], GShard [29], Switch [16], ST-MoE [70], DeepSpeed-MoE [41], Mixtral [23], DeepSeek-MoE [14], and OLMoE [35] together established that MoE expert dispatch dominates the training step once active parameters exceed a few hundred million. Expert-choice routing [69] fixes per-expert receive counts and removes per-step host-side count computation from the AllToAllV path; we use it in the end-to-end OLMoE evaluation. On Llama [57, 58], the PP cross-stage send/recv + TP MLP + FSDP weight-prefetch strategy we evaluate matches the standard training recipe. Trainium practitioners typically issue one collective per microbatch (e.g. one TP-MLP all-reduce per microbatch for a training step of M microbatches uses M all-reduce calls). We show that, when the per-microbatch payloads can share an underlying buffer, the same logical strategy can also be implemented by issuing a single collective per training step that covers all M microbatches at once — trading M launch costs for one. Whether this trade is profitable depends on the device (a larger payload may take longer to compile or exceed memory limits), so we let the search make that choice per problem rather than fixing it by hand.

7 Conclusion

We presented a search loop that finds faster collective-communication strategies on top of a closed-source vendor library, where two structural properties make optimization hard: (C1) no visibility into the runtime’s behavior, and (C2) no low-level control over how primitives are scheduled. For (C1), we have an LLM agent build a cost-model simulator on the target deployment itself, calibrating its internal numbers against the actual device and model shape — so the simulator stays workload-faithful. For (C2), we accept the vendor API as fixed and search one layer above it, over which primitives to call, in what order, and with what tensor pre- and post-processing. On AWS Trainium the deployed strategies deliver 1.40× end-to-end on OLMoE-10B (with algorithm-equivalent descent on real OpenWebText) and 3.24× on Llama-7B over an internal-AWS-optimized production baseline. An ablation that hides the simulator from the LLM collapses the OLMoE speedup back to 1.00×, confirming that the on-device-calibrated simulator — not the LLM — is what picks the strategy that actually wins at real training runs.

References

- [1] Amazon Web Services. 2024. AWS Neuron Documentation: Trainium Architecture. <https://awsdocs-neuron.readthedocs-hosted.com/en/latest/about-neuron/arch/neuron-hardware/trainium.html>. Accessed 2026-05-15.
- [2] Amazon Web Services. 2024. AWS Trainium2 and Trn2 Instances for Generative AI. AWS Product Documentation. <https://aws.amazon.com/ai/machine-learning/trainium/>
- [3] Amazon Web Services. 2024. Training with Tensor Parallelism (NeuronX Distributed). AWS Neuron SDK Documentation. <https://awsdocs-neuron.readthedocs-hosted.com/en/latest/libraries/neuronx-distributed/tutorials/training.html>
- [4] Amazon Web Services. 2024. Tutorials for NeuronX Distributed. AWS Neuron SDK Documentation. <https://awsdocs-neuron.readthedocs-hosted.com/en/latest/libraries/neuronx-distributed/tutorials/index.html>
- [5] Anonymous. 2025. STARK: Strategic Team of Agents for Refining Kernels. <https://arxiv.org/abs/2510.16996> arXiv:2510.16996.
- [6] Anonymous. 2026. K-Search: LLM Kernel Generation via Co-Evolving Intrinsic World Model. <https://arxiv.org/abs/2602.19128> arXiv:2602.19128.
- [7] Anthropic. 2025. Claude Code: An interactive CLI agent for code. <https://www.anthropic.com/claude-code>
- [8] Zixian Cai, Zhengyang Liu, Saeed Maleki, Madanlal Musuvathi, Todd Mytkowicz, Jacob Nelson, and Olli Saarikivi. 2021. Synthesizing Optimal Collective Algorithms. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*. ACM, 62–75. doi:10.1145/3437801.3441620
- [9] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).
- [10] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Meghan Cowan, Haichen Shen, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [11] Tianqi Chen, Lianmin Zheng, Eddie Yan, Ziheng Jiang, Thierry Moreau, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. Learning to Optimize Tensor Programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 31. <https://arxiv.org/abs/1805.08166>
- [12] Meghan Cowan, Saeed Maleki, Madanlal Musuvathi, Olli Saarikivi, and Yifan Xiong. 2023. MSCCLang: Microsoft Collective Communication Language. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 502–514. doi:10.1145/3575693.3575724 Extends MSCCL; arXiv:2201.11840.
- [13] Stanford CS. 2025. Astra: A Multi-Agent System for GPU Kernel Performance Optimization. <https://arxiv.org/abs/2509.07506> arXiv:2509.07506.
- [14] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. 2024. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. <https://arxiv.org/abs/2401.06066>
- [15] Alhussein Fawzi, Matej Balog, Aja Huang, Thomas Hubert, Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, et al. 2022. Discovering Faster Matrix Multiplication Algorithms with Reinforcement Learning. *Nature* 610 (2022), 47–53.
- [16] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.
- [17] Aaron Gokaslan, Vanya Cohen, Ellie Pavlick, and Stefanie Tellex. 2019. OpenWebText Corpus. <http://Skylion007.github.io/OpenWebTextCorpus>.
- [18] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. *arXiv preprint arXiv:2401.14196* (2024).
- [19] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, Hyoungho Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. 2019. GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [20] Abhinav Jangda, Jun Huang, Guodong Liu, Amir Hossein Nodehi Sabet, Saeed Maleki, Youshan Miao, Madanlal Musuvathi, Todd Mytkowicz, and Olli Saarikivi. 2022. Breaking the Computation and Communication Abstraction Barrier in Distributed Machine Learning Workloads. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 402–416. doi:10.1145/3503222.3507778
- [21] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. 2019. TASO: Optimizing Deep Learning Computation with Automatic Generation of Graph Substitutions. In *ACM Symposium on Operating Systems Principles (SOSP)*.
- [22] Zhihao Jia, Matei Zaharia, and Alex Aiken. 2019. Beyond Data and Model Parallelism for Deep Neural Networks. In *Proceedings of Machine Learning and Systems (MLSys)*.
- [23] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, et al. 2024. Mixtral of Experts. *arXiv preprint arXiv:2401.04088* (2024).
- [24] Yimin Jiang, Yibo Zhu, Chang Lan, Bairen Yi, Yong Cui, and Chuanxiong Guo. 2020. A Unified Architecture for Accelerating Distributed DNN Training in Heterogeneous GPU/CPU Clusters. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [25] Levente Kocsis and Csaba Szepesvári. 2006. Bandit Based Monte-Carlo Planning. In *Machine Learning: ECML 2006*. Springer, 282–293.
- [26] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2023. Reducing Activation Recomputation in Large Transformer Models. In *Proceedings of Machine Learning and Systems (MLSys)*.
- [27] Robert Tjarko Lange et al. 2025. ShinkaEvolve: Towards Open-Ended and Sample-Efficient Program Evolution. *arXiv preprint arXiv:2509.19349* (2025). <https://arxiv.org/abs/2509.19349>
- [28] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2021. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. In *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*.
- [29] Dmitry Lepikhin, Hyoungho Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2021. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. In *International Conference on Learning Representations (ICLR)*.
- [30] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, and Soumith Chintala. 2020. PyTorch Distributed: Experiences on Accelerating Data Parallel Training. *Proceedings of the VLDB Endowment* 13, 12 (2020).
- [31] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, et al. 2022. Competition-Level

- Code Generation with AlphaCode. *Science* 378, 6624 (2022), 1092–1097.
- [32] Hao Liu, Matei Zaharia, and Pieter Abbeel. 2024. Ring Attention with Blockwise Transformers for Near-Infinite Context. In *International Conference on Learning Representations (ICLR)*.
- [33] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [34] Daniel J. Mankowitz, Andrea Michi, Anton Zhernov, Marco Gelmi, Marco Selvi, Cosmin Paduraru, Edouard Leurent, et al. 2023. Faster Sorting Algorithms Discovered Using Deep Reinforcement Learning. *Nature* 618 (2023), 257–263.
- [35] Niklas Muennighoff, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Jacob Morrison, Sewon Min, Weijia Shi, Pete Walsh, Oyvind Tafjord, Nathan Lambert, Yuling Gu, Shane Arora, Akshita Bhagia, Dustin Schwenk, David Wadden, Alexander Wettig, Binyuan Hui, Tim Dettmers, Douwe Kiela, Ali Farhadi, Noah A. Smith, Pang Wei Koh, Amanpreet Singh, and Hannaneh Hajishirzi. 2025. OLMoE: Open Mixture-of-Experts Language Models. In *International Conference on Learning Representations (ICLR)*. arXiv:2409.02060.
- [36] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. 2019. PipeDream: Generalized Pipeline Parallelism for DNN Training. In *ACM Symposium on Operating Systems Principles (SOSP)*.
- [37] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. 2021. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.
- [38] Alexander Novikov et al. 2025. AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery. *arXiv preprint arXiv:2506.13131* (2025). <https://arxiv.org/abs/2506.13131>
- [39] NVIDIA Corporation. 2024. NCCL: NVIDIA Collective Communications Library. <https://github.com/NVIDIA/nccl>. Accessed 2026-05-15.
- [40] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*.
- [41] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. 2022. DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale. In *International Conference on Machine Learning (ICML)*.
- [42] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.
- [43] Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. 2020. ASTRA-SIM: Enabling SW/HW Co-design Exploration for Distributed DL Training Platforms. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*.
- [44] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*.
- [45] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. 2021. ZeRO-Offload: Democratizing Billion-Scale Model Training. In *USENIX Annual Technical Conference (ATC)*.
- [46] Meta Research. 2025. LLM-Guided Evolutionary Kernel Optimization: From Research to Production Kernels. <https://mlai.blog/2025-12-20-llm-kernel-optimization>
- [47] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. 2024. Mathematical Discoveries from Program Search with Large Language Models. *Nature* 625, 7995 (2024), 468–475. doi:10.1038/s41586-023-06924-6
- [48] Stuart Russell and Peter Norvig. 2010. *Artificial Intelligence: A Modern Approach* (3 ed.). Prentice Hall.
- [49] Alexander Sergeev and Mike Del Balso. 2018. Horovod: Fast and Easy Distributed Deep Learning in TensorFlow. *arXiv preprint arXiv:1802.05799* (2018).
- [50] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, Olli Saarikivi, and Rachee Singh. 2023. TACCL: Guiding Collective Algorithm Synthesis using Communication Sketches. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI '23)*. USENIX Association, 593–612. <https://www.usenix.org/conference/nsdi23/presentation/shah>
- [51] Noam Shazeer. 2020. GLU Variants Improve Transformer. *arXiv:2002.05202*.
- [52] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In *International Conference on Learning Representations (ICLR)*.
- [53] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [54] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *arXiv preprint arXiv:1909.08053* (2019).
- [55] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing* (2024).
- [56] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*. ACM, 10–19. doi:10.1145/3315508.3329973
- [57] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, et al. 2023. LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971* (2023).
- [58] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288* (2023).
- [59] Nicolas Vasilache, Oleksandr Zinenko, Theodoros Theodoridis, Priya Goyal, Zachary DeVito, William S. Moses, Sven Verdoolaege, Andrew Adams, and Albert Cohen. 2018. Tensor Comprehensions: Framework-Agnostic High-Performance Machine Learning Abstractions. *arXiv preprint arXiv:1802.04730* (2018).
- [60] Guanhua Wang, Shivaram Venkataraman, Amar Phanishayee, Nikhil Devanur, Jorgen Thelin, and Ion Stoica. 2020. Blink: Fast and Generic Collectives for Distributed ML. In *Proceedings of Machine Learning and Systems (MLSys)*, Vol. 2. 172–186.
- [61] Xiaoyang Wang et al. 2025. SimAI: Unifying Architecture Design and Performance Tuning for Large-Scale Large Language Model Training

- with Scalability and Precision. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI '25)*.
- [62] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. Self-Instruct: Aligning Language Models with Self-Generated Instructions. In *Annual Meeting of the Association for Computational Linguistics (ACL)*.
- [63] William Won, Jaehong Heo, Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. 2023. ASTRA-sim2.0: Modeling Hierarchical Networks and Disaggregated Systems for Large-Model Training at Scale. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*.
- [64] Guanbin Xu, Zhihao Le, Yinhe Chen, Zhiqi Lin, Zewen Jin, Youshan Miao, and Cheng Li. 2025. AutoCCL: Automated Collective Communication Tuning for Accelerating Distributed and Parallel DNN Training. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. USENIX Association, 667–683. <https://www.usenix.org/conference/nsdi25/presentation/xu-guanbin>
- [65] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [66] Biao Zhang and Rico Sennrich. 2019. Root Mean Square Layer Normalization. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [67] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, et al. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. *Proceedings of the VLDB Endowment* 16, 12 (2023).
- [68] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. 2020. Ansor: Generating High-Performance Tensor Programs for Deep Learning. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [69] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M. Dai, Zhifeng Chen, Quoc V. Le, and James Laudon. 2022. Mixture-of-Experts with Expert Choice Routing. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35. <https://arxiv.org/abs/2202.09368>
- [70] Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. 2024. ST-MoE: Designing Stable and Transferable Sparse Expert Models. In *International Conference on Learning Representations (ICLR)*. arXiv:2202.08906.